Transformer-Based Representation Learning for Manufacturing Process Chains Under Missing Data and Data Scarcity

Francesco Scipione[1†,2], Bekim Berisha[1,2], Selcuk Mentese[3], Uwe Kramer[3], Robin Wieland[3], Dirk Mohr[1*]

[1]*Department of Mechanical and Process Engineering, Swiss Federal Institute of Technology (ETH), Zurich, Switzerland;*
[†]*fscipion@ethz.ch;* [*]*dmohr@ethz.ch;*
[2]*Inspire AG, Zurich, Switzerland*
[3]*Dyconex AG, , Bassersdorf, Switzerland*

Abstract. Contemporary manufacturing relies on complex process chains, where product quality and reliability emerge from interactions across multiple fabrication steps rather than from isolated stages. Modeling such chains, therefore, requires representations that can capture long-range, cross-stage dependencies under realistic constraints such as missing data and limited sample sizes. In this work, we formulate manufacturing process modeling as a sequence-learning problem and introduce a transformer-based framework that represents entire process chains as sequences of parameter tokens. Instead of learning application-specific features, the proposed model learns a general representation of manufacturing processes directly from heterogeneous process data. We evaluate the proposed approach on both a controlled synthetic benchmark and a real-world industrial dataset from multilayer printed circuit board manufacturing, which we publicly release. Using these datasets, we study performance degradation under missing data and data scarcity, two conditions that frequently arise in industrial settings. We further demonstrate the flexibility of the framework by incorporating a physics-informed supervisory signal and by improving the prediction of rare failure cases through curriculum learning and online inference. Our results indicate that transformer-based representations can effectively model entire manufacturing process chains and adapt to different supervisory signals, while remaining robust under realistic data limitations.

Keywords: Transformer, manufacturing, process chain, statistical modeling, machine learning

1. Introduction

Models of manufacturing processes are essential for describing and predicting the relationships between process parameters, system behavior, product quality, and final performance. As such, they provide the foundation for process understanding, optimization, monitoring, and control, and are therefore key enablers of efficient, robust, and intelligent manufacturing systems. Modern manufacturing lines generate heterogeneous data throughout the process chain, including set-point parameters, in situ sensor measurements, machine states, operator annotations, and timestamps. Despite this richness, most analytical pipelines still model individual process steps in isolation or rely on aggregated batch-level statistics (Weichert et al., 2019). Such approaches neglect cross-stage dependencies, in which interactions between temporally distant process steps accumulate and ultimately determine product quality and reliability. Capturing these effects requires a modeling paradigm that treats manufacturing as a sequential, multivariate system rather than as a collection of independent operations. The goal is to learn compact and actionable representations that preserve the structure of the process chain.

From a modeling perspective, manufacturing process chains share important structural similarities with sequential data in natural language, where the interpretation of individual elements depends on their position and on the surrounding sequence context. By analogy, a fabrication step can be represented as a token, and the complete process chain as an ordered sequence of such tokens. Transformers are well-suited to this setting because they learn context-dependent representations through attention mechanisms, allowing each process step to be interpreted in light of the full manufacturing history. This is particularly important in manufacturing because the outcome of a given step often depends not only on local process settings, but also on decisions and conditions introduced much earlier in the production flow. Such long-range interdependencies are generally unknown a priori and may span multiple stages, making them difficult to capture with local or stage-wise models.

A second challenge arises from the repeated execution of nominally identical process steps, such as drilling operations performed on different regions of the product. Although

such steps may share identical parameter values, their effect on the final product can differ substantially depending on their position in the process chain and the surrounding process context. A suitable representation must therefore distinguish repeated process settings through contextual information rather than treating them as interchangeable inputs. By encoding manufacturing steps as tokens enriched with stage-specific, positional, and occurrence-dependent attributes, a Transformer model can learn context-sensitive representations and capture how identical settings yield different outcomes under varying manufacturing conditions.

Beyond representational expressivity, practical deployment in industrial settings imposes additional requirements. Real-world manufacturing datasets are often incomplete due to sensor failures, logging inconsistencies, or asynchronous data acquisition. In addition, although modern production systems generate large volumes of raw data, labeled datasets suitable for supervised learning are often smaller than those commonly used in other domains. Robust manufacturing models must therefore be evaluated not only for predictive performance, but also for their ability to handle missing information and data-scarce regimes.

In this work, we propose a Transformer-based framework for learning general representations of manufacturing process chains by modeling process parameters as feature-enriched tokens in an ordered sequence. The framework is agnostic to the specific process structure and can therefore be applied to heterogeneous manufacturing pipelines. We systematically analyze its performance under two realistic industrial constraints, missing data and limited dataset size, using a controlled synthetic dataset that enables targeted corruption and scaling of the training data. We further validate the approach on a real-world dataset from multilayer printed circuit board manufacturing, demonstrating competitive predictive performance for reliability testing. Finally, the case study illustrates the flexibility of the proposed representation for addressing practical downstream tasks, including the improved learning of rare failure cases and the integration of physics-informed supervisory signals.

2. Related work

The adoption of statistical modeling to improve various aspects of industrial manufacturing dates back at least to the beginning of the twentieth century, inspiring the seminal work of Shewart (Shewhart and Deming, 1939), which statistically formalized manufacturing and highlighted the advantages of *control* in production, anticipating the field of Statistical Process Control (SPC). Manufacturing is inherently multi-stage, and randomness propagates downstream along the process chain: a global, multi-stage modeling paradigm is necessary. Two main approaches are viable, physics- or data-driven. For the former, a cornerstone is the Stream of Variation (SoV) theory, which employs State-Space (SS) models to encode multi-stage manufacturing. Consider the vector X describing the errors in the manufactured structure. As the structure moves through the stages of production, X is updated in the classical SS formalism, where the time is replaced by the stage index k, according to the stage uncertainty U_k and the error from the previous stage, i.e. $X_k = A_{k-1}X_{k-1} + B_k U_k$. Each stage produces noisy (ε_k) observations $Y_k = C_k X_k + \varepsilon_k$ or just a final observation at the end of the manufacturing process. The method quantifies the uncertainty of Y given U_k and ε_k for all k. The matrices A, B, C must be handcrafted for each stage, using deep knowledge and engineering expertise in the specific application domain. (Huang and Shi, 2004) present an extension of SoV in a more general, serial-parallel, multi-stage industrial process, demonstrating how downstream randomness propagates across different process routes. (Zhou et al., 2003) proposes a mixed linear SS model to capture both deterministic and random effects. The authors further study the concept of model *identifiability* in the root-cause analysis problem, rebaptizing it as "diagnosability".

To overcome the limitations of the physics-based formulation, researchers shifted to data-driven methods for modeling complex multi-stage manufacturing processes. A relevant model class are Markov models, which consider the manufacturing process as a sequence, (similar to SS models) $X_1, \dots, X_T$, where $X_{t+1} \perp X_1, \dots, X_{t-1} | X_t$ (Markov property). Each stage X_t can transition from one state to another with some transition probabilities. States can represent adding/repairing the defect, for example, with the

associated probability of degrading/repairing the structure. The objective is to learn the transition probability from historical data and estimate the final quality of the manufactured structure (Du et al., 2018, 2015). Note how the model assumption has been relaxed to a simple probabilistic state transition, but to calculate the final quality, we still need to assume the same multi-stage protocol. As manufacturing and the respective data grow in complexity, more flexible paradigms are desired.

Machine Learning (ML) encompasses a class of non-parametric models and algorithms designed to learn complex relationships in data, with the primary objective of making predictions on unseen input data. By their non-parametric design, ML models such as the Multi-Layer Perceptron (MLP) are highly flexible and can theoretically approximate any non-linear function, as stated by the renowned Universal Approximation Theorem (Lu and Lu, n.d.). Early adoption of ML in manufacturing was limited to single-stage prediction tasks (Mok and Kwong, 2002), in which the interplay among process parameters, product geometry, and the final quality metric is too complex to capture with an adequate parametric form. As previously seen, multi-stage manufacturing finds its natural problem formulation in sequential tasks, and ML methods proved to be a powerful tool in a particularly complex sequential task: natural language processing (NLP); mainly boosted by the Transformer architecture. The Transformer (Vaswani et al., 2023) addressed key limitations of recurrent neural networks in sequence modeling, particularly sequential training and vanishing gradients, by replacing recurrence with attention-based parallel processing. This design enabled scalable learning of long-range dependencies and established Transformers as a dominant framework in natural language processing, leading to models such as BERT (Devlin et al., 2019) and the GPT family (Brown et al., 2020; Radford et al., n.d.). Beyond language, the Transformer has also been successfully adapted to other domains. In computer vision, for example, the Vision Transformer demonstrated that images can be modeled effectively as sequences of patches without relying on convolutional inductive biases (Dosovitskiy et al., 2021). In time-series analysis, Transformer variants such as the Time Series Transformer (Li et al., 2020) and Informer (Zhou et al., 2021) extended attention-based sequence modeling to long-horizon forecasting. The Informer, in particular, improved efficiency by introducing a sparse

attention mechanism and non-autoregressive decoding for sequence prediction. More recently, Spacetimeformer (Grigsby et al., 2023) generalized Transformer modeling to multivariate spatiotemporal data by flattening variables and time steps into a unified token sequence. This formulation is particularly relevant for manufacturing, where heterogeneous variables evolve across multiple stages and scales. The present work builds on this idea by extending tokenization beyond temporal forecasting to full manufacturing process chains, in which process parameters, rather than time steps alone, form the fundamental sequence elements.

Transformer-based models have recently been explored in several manufacturing applications, primarily for analyzing sequential data generated within individual process stages. Representative examples include predictive maintenance, where attention mechanisms have been used to model tool wear and degradation from sensor time series (Liu et al., 2020), and anomaly detection, where Transformer architectures have been combined with adversarial or convolutional components to identify irregular patterns in long process signals (Orabi et al., 2024; Cai et al., 2023). A smaller body of work has investigated Transformers for optimization and control tasks, leveraging their expressivity and computational efficiency for real-time decision making in industrial systems (Gallup et al., 2024). Other variants, such as dual-attention mechanisms that separately model temporal and variable dimensions, have been proposed to better capture multivariate structure (Wang et al., 2024). Although these approaches improve representation capacity, their complexity often scales with the number of process variables, which can limit applicability in large and heterogeneous manufacturing systems.

Overall, existing Transformer-based methods in manufacturing predominantly operate at the level of individual processes or short temporal windows. As a result, they largely overlook the propagation of information across multiple fabrication stages, which is essential for modeling end-to-end manufacturing performance. In addition, robustness to missing data and limited dataset size—two pervasive characteristics of industrial datasets—is often treated only implicitly or addressed through standard imputation procedures. In contrast, the present work focuses on learning representations of entire manufacturing process chains while explicitly accounting for missing values and data

scarcity. To this end, we build on the masking-based missing-data treatment proposed by (Caruso et al., 2024) and extend it to process-chain modeling with self- and cross-attention over manufacturing tokens.

3. Manufacturing process model

The manufacturing process model aims to establish the relationship between the characteristics of the manufacturing process, as described by an L_x-long indexed list of p_x-dimensional process descriptors,

$$\left(\mathbf{X}^{(1)}, \mathbf{X}^{(2)}, \ldots, \mathbf{X}^{(L_x)}\right), \tag{1}$$

and a sequence of product quality descriptors (e.g. a measurement signal over time in a product performance test), denoted by an L_y-long sequence of p_y-dimensional vectors,

$$\left(\mathbf{Y}^{(1)}, \mathbf{Y}^{(2)}, .., \mathbf{Y}^{(L_y)}\right). \tag{2}$$

We consider the process descriptors as tokens $\mathbf{X}^{(\mathrm{i})}$, with each token featuring a value and context features (type of process, variable descriptor, etc.). For example, for $p_x = 4$, an input token with the features (1) value, (2) variable, (3) process and (4) timestamp may resemble

$$\mathbf{X}^{(\mathrm{i})} = (185.0, \text{"temperature"}, \text{"curing"}, \text{"22.6.2025"}). \tag{3}$$

The length L_X is determined by the number of non-redundant process descriptors that are needed to describe the complete manufacturing process.

Similarly, an output token with the $p_y = 3$ features, for example, (1) value, (2) variable, (3) time, may be written as

$$\mathbf{Y}^{(\mathrm{i})} = (0.7, \text{"resistance_terminal_A"}, 53). \tag{4}$$

The output length L_Y is then given by the number of points that discretize the output signal history.

Formally, the input-to-output relationship describes a list-to-list mapping,

$$f: \left(\mathbf{X}^{(1)}, \mathbf{X}^{(2)}, \ldots, \mathbf{X}^{(L_x)}\right) \rightarrow \left(\mathbf{Y}^{(1)}, \mathbf{Y}^{(2)}, .., \mathbf{Y}^{(L_y)}\right), \tag{5}$$

which may also be seen as a sequence-to-sequence mapping, except that the order of the tokens is not assumed to define a process-relevant order. Instead, temporal and process-ordering information, if applicable, needs to be encoded explicitly through token features. In the sequel, we approximate this mapping using a transformer-based encoder-decoder model. A block schematic is provided in Figure 1.

3.1.1 Embedding block

The embedding block transforms the semantic token description of the manufacturing process into a set of real-valued vectors, $\mathbf{x}^{(\mathrm{i})} \in R^d$, where d denotes the dimension of the transformer model input space. Formally, a model input vector $\mathbf{x}^{(i)}$ is obtained from a semantic token $\mathbf{X}^{(i)}$ by applying suitable embedding functions to the individual token features. Categorical features in string format are first mapped to integer indices using predefined dictionaries. These indices are then mapped to real-valued vectors in R^d by embedding tables. Numerical features, such as measured process values or timestamps, are mapped to vectors in R^d using appropriate numerical embedding functions, such as linear projections, sinusoidal encodings, or Time2Vec.

For an individual scalar feature $\mathrm{X}_j^{(i)}$, the feature-to-vector mapping reads

$$\mathbf{X}_j^{(i)} \rightarrow \boldsymbol{\varphi}_j[\mathrm{X}_j^{(i)}] \,. \tag{6}$$

The embedded input vector $\boldsymbol{x}^{(i)} \in R^d$ is then obtained by summing all feature embeddings,

$$\boldsymbol{x}^{(i)} = \boldsymbol{\varphi}_1[\mathrm{X}_1^{(i)}] + \boldsymbol{\varphi}_2[\mathrm{X}_2^{(i)}] + \cdots + \boldsymbol{\varphi}_{p_x}[\mathrm{X}_{p_x}^{(i)}]. \tag{7}$$

As detailed in Section 6.2, we use a combination of fixed and learnable embeddings. After completing the embedding operation for all L_x input tokens, we obtain the model input sequence

$$\{\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, .., \mathbf{x}^{(L_x)}\} \quad \text{with} \quad \mathbf{x}^{(i)} \in R^d. \tag{8}$$

This sequence of real-valued model-space vectors is then processed by the transformer model.

3.1.2 Encoder block

The vector representation $\left(\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, .., \mathbf{x}^{(L_x)}\right)$ of the input tokens enters the encoder block of the transformer. As the encoder block is composed by several identical stacked encoder layers, we describe the functionality of one layer.

Each input vector is first normalized independently using Layer Normalization,

$$\tilde{\boldsymbol{x}}^{(i)} = \text{LayerNorm}[\mathbf{x}^{(i)}]. \tag{9}$$

This normalization operation across the d components of each $\boldsymbol{x}_i$ produces vectors $\tilde{\boldsymbol{x}}^{(i)}$ of zero mean and unit variance.

Following a standard self-attention formulation, so-called keys $\boldsymbol{k}_{\mathrm{i}} \in R^{d_{\mathrm{QK}}}$, queries $\boldsymbol{q}_{\mathrm{i}} \in R^{d_{\mathrm{QK}}}$, and values $\boldsymbol{v}_{\mathrm{i}} \in R^{\mathrm{d}}$ are computed from the normalized input vectors through linear transformation,

$$\boldsymbol{k}^{(i)} = \mathbf{W}_K \tilde{\boldsymbol{x}}^{(i)}, \tag{10}$$

$$\boldsymbol{q}^{(i)} = \mathbf{W}_Q \tilde{\boldsymbol{x}}^{(i)}, \tag{11}$$

$$\boldsymbol{v}^{(i)} = \mathbf{W}_V \tilde{\boldsymbol{x}}^{(i)}, \tag{12}$$

where $\mathbf{W}_K$, $\mathbf{W}_Q \in R^{d_{QK} \times d}$ and $\mathbf{W}_V \in R^{d \times d}$ are encode block-specific learnable matrices. The model computes compatibility scores between all query-key pairs,

$$s_{ij} = \frac{\boldsymbol{q}^{(i)} \cdot \boldsymbol{k}^{(j)}}{\sqrt{d_{QK}}}, \quad i, j = 1, \ldots, L_x, \tag{13}$$

with the scaling factor $\sqrt{d_{QK}}$ stabilizing the magnitude of the dot product. The scores are converted into attention weights α_{ij} by applying the softmax function over the index j,

$$\alpha_{ij} = \text{Softmax}[s_{ij}] \coloneqq \frac{\exp\,[s_{ij}]}{\sum_l \exp\,[s_{il}]}. \tag{14}$$

Thus, for each i, the sum of α_{ij} over $j = 1, \ldots, L_x$ equals one. The output of one attention head is then obtained as the weighted sum of the value vectors,

$$\boldsymbol{h}^{(i)} = \sum_{j=1}^{L_X} \alpha_{ij} \boldsymbol{v}^{(j)} \,. \tag{15}$$

In the multi-head attention (MHA), several heads are concatenated and combined by an appropriate output projection. Let $\boldsymbol{h}^{k,(i)}$ be the output of the k-th head, parametrized by the learnable maps $\mathbf{W}_K^k, \mathbf{W}_Q^k, \mathbf{W}_V^k$. The MHA output, for H heads, is given by

$$\boldsymbol{h}^{\mathrm{MHA}(i)} = \mathrm{concat}\left(\boldsymbol{h}^{1,(i)}, \ldots, \boldsymbol{h}^{H,(i)}\right)\mathbf{W}_{\mathrm{O}}, \tag{16}$$

where $\mathbf{W}_{\mathrm{O}} \in R^{dH \times d}$.

Each latent output vector $\boldsymbol{h}^{\mathrm{MHA}(i)}$ therefore contains a mixture of information from all input tokens. $\boldsymbol{h}^{\mathrm{MHA}(i)}$ is added to the residual stream,

$$\boldsymbol{z}^{(i)} = \boldsymbol{x}^{(i)} + \boldsymbol{h}^{\mathrm{MHA}(i)}, \tag{17}$$

before passing the result through a Multilayer Perceptron (MLP), which applies a nonlinear transformation along the d dimension.

$$\boldsymbol{x}_{out}^{(i)} = MLP(\boldsymbol{z}^{(i)}). \tag{18}$$

The sequence $\left(\boldsymbol{x}_{out}^{(1)}, \boldsymbol{x}_{out}^{(2)}, \ldots, \boldsymbol{x}_{out}^{(L_X)}\right)$ forms the output of one encoder block and can be passed in subsequent encoder and/or decoder blocks. Let f_{E_i} be the function of the i-th encoder block, for D stacked encoder blocks, the final encoder output is given by the composition

$$\boldsymbol{x}_{out,D}^{(i)} = f_{E_D} \circ \ldots \circ f_{E_2} \circ f_{E_1}(\boldsymbol{x}^{(i)}). \tag{19}$$

3.1.3 Decoder block

Aside from the encoded input tokens, the decoder will also require the context features of the output tokens as input. These features inform the decoder about the context for the

output token values it needs to predict. Assuming that the value is the first feature of an output token $\mathbf{Y}^{(i)}$, the embedding of all output context features is computed based on the sum over the features 2 through p_Y only,

$$\boldsymbol{y}^{(i)} = \boldsymbol{\varphi}_2[\mathbf{Y}_2^{(i)}] + \cdots + \boldsymbol{\varphi}_{p_y}[\mathbf{Y}_{p_y}^{(i)}]. \tag{20}$$

$\boldsymbol{y}^{(i)}$ serves as input to the decoder block, while the feature $\mathbf{Y}_1^{(i)}$, the value feature, is the quantity to be predicted by the transformer model. As for the encoder, the embedding maps the tokens $\mathbf{Y}^{(i)}$ to a d-dimensional real-valued vectors $\boldsymbol{y}^{(i)}$.

Except for the input embeddings, the self-attention line of the decoder is the same as that of the input encoder layer (Section 3.1.3). The self-attention produces a latent output $\boldsymbol{u}^{(i)}$ based on the embedding $\boldsymbol{y}^{(i)}$, analogously to the transformer's input encoder that produces the latent output $\boldsymbol{z}^{(i)}$ based on the embedding $\boldsymbol{x}^{(i)}$.

The vector sequences $\left(\boldsymbol{x}_{out}^{(1)}, \boldsymbol{x}_{out}^{(2)}, \ldots, \boldsymbol{x}_{out}^{(L_X)}\right)$ and $\left(\boldsymbol{u}^{(1)}, \boldsymbol{u}^{(2)}, \ldots, \boldsymbol{u}^{(L_Y)}\right)$ form the input of the cross-attention. $\boldsymbol{u}^{(i)}$ is normalized by a normalization block

$$\tilde{\boldsymbol{u}}^{(i)} = \mathrm{LayerNorm}[\boldsymbol{u}^{(i)}] \tag{21}$$

and passed to the queries of a cross-attention layer,

$$\boldsymbol{q}^{(i)} = \mathbf{W}_Q \tilde{\boldsymbol{u}}^{(i)}. \tag{22}$$

The encoder outputs $\boldsymbol{x}_{out,\mathrm{D}}^{(i)}$ enter as input for the keys and values of the cross-attention layer,

$$\boldsymbol{k}^{(i)} = \mathbf{W}_K \boldsymbol{x}_{out,\mathrm{D}}^{(i)}, \tag{23}$$

$$\boldsymbol{v}^{(i)} = \mathbf{W}_V \boldsymbol{x}_{out,\mathrm{D}}^{(i)}, \tag{24}$$

The equations (13), (14), (15) are then used to compute scores, attention weights, and final cross-attention head output.

As for self-attention, H' cross-attention heads can be concatenated and combined by the multi-head output map $\mathbf{W}_\mathrm{O}$ using equation (16), obtaining the vector $\boldsymbol{w}^{\mathrm{MHA}(i)}$.

which is then added to the residual stream

$$\boldsymbol{t}^{(i)} = \boldsymbol{u}^{(i)} + \boldsymbol{w}^{\mathrm{MHA}(i)}. \tag{25}$$

The result is passed to a non-linear MLP block, which yields the vector

$$\boldsymbol{y}_{out}^{(i)} = MLP(\boldsymbol{t}^{(i)}). \tag{26}$$

When stacking multiple decoder layers, the input of the k-th layer f_k is given by $\boldsymbol{u}^{(i)} = \boldsymbol{y}_{out,k-1}^{(i)}$ while its output is $\boldsymbol{y}_{out,k}^{(i)} = \boldsymbol{y}_{out}^{(i)}$. Each decoder then defines a sequence-to-sequence mapping

$$f_k\colon (\boldsymbol{y}_{out,k-1}^{(i)}, \boldsymbol{x}_{out,D}^{(i)}) \rightarrow \boldsymbol{y}_{out,k}^{(i)} \tag{27}$$

For D' stacked decoder layers, the final decoder block output is given by the composition

$$\boldsymbol{y}_{out,D'}^{(i)} = f_{D'} \circ \ldots \circ f_2 \circ f_1(\boldsymbol{y}^{(i)}, \boldsymbol{x}_{out,D}^{(i)}). \tag{28}$$

3.1.4 Final feed-forward block

The decoder output $\boldsymbol{y}_{out,D'}^{(i)}$ still has a d-dimensional representation. The final feedforward layer $f_{FF}\colon R^d \rightarrow R^o$, where o is the output dimension. For supervised learning, the output dimension corresponds to the dimension of the supervisory signal.

$$\hat{\boldsymbol{y}}^{(i)} = f_{FF}(\boldsymbol{y}_{out,D'}^{(i)}). \tag{29}$$

3.1.5 Handling of missing feature information

Missing or incomplete tokens are common in manufacturing datasets and can arise from sensor failures, logging issues, or padding of variable-length sequences. When

missing entries are naively imputed (e.g. set to zero), attention mechanisms may incorrectly treat them as informative inputs, leading to biased representations and unstable training. To address this issue, we adopt the Not-Another-Imputation Method as Not-Available-Information Mask (NAIM), (Caruso et al., 2024), which prevents missing values from contributing to attention computations. The idea is to block both information flow and gradient propagation associated with missing tokens, ensuring that the model learns only from valid observations.

To handle both self-attention and cross-attenion, we distinguish between a binary mask $\boldsymbol{B}^K \in \{0,1\}^{L_K}$ for the key tokens and a binary mask $\boldsymbol{B}^Q \in \{0,1\}^{L_Q}$ for the query tokens, which are identical in the case of self-attention. The entry $B_i = 1$ indicates that the token i is present and contains valid information, whereas $B_i = 0$ indicates that it is missing or padded.

From the binary key validity mask, we define the additive key mask

$$M_{ij}^K = \begin{cases} 0, & if \quad B_j^K = 1 \\ -\infty, & if \quad B_j^K = 0 \end{cases} \tag{30}$$

for $i = 1, \dots, L_Q$ and $j = 1, \dots, L_K$. The key mask M_{ij}^K is added to the attention scores for the query-key pair $(\boldsymbol{q}^{(i)}, \boldsymbol{k}^{(j)})$ before the softmax operation, while the binary query mask B_i^Q is directly used as a multiplier,

$$\alpha_{ij} = B_i^Q \mathrm{Softmax}[s_{ij} + M_{ij}^K]. \tag{31}$$

As a consequence, we have $\alpha_{ij} = 0$ whenever a query $\boldsymbol{q}^{(i)}$ or key $\boldsymbol{k}^{(j)}$ is invalid.

3.1.6 Physics-Informed Simulator

In addition to standard sequence-to-sequence prediction, we include a lightweight simulator that rolls out the target from a small set of learned physical variables. Formally, the input-output relation of equation () does not change. The relation is described as

$$f: \left(\mathbf{X}^{(1)}, \mathbf{X}^{(2)}, \dots, \mathbf{X}^{(L_x)}\right) \rightarrow \left(\mathbf{Y}^{(1)}, \mathbf{Y}^{(2)}, \dots, \mathbf{Y}^{(L_s)}\right), \tag{32}$$

where L_s is the number of variables needed by the simulator. For example, a token describing the variable of the simulator could be

$$\mathbf{Y}^{(\mathrm{i})} = (0.7, \text{"void growth rate"}, \text{"material A"}). \tag{33}$$

The physics-based model is intended as an illustrative supervisory signal rather than a high-fidelity simulator, and no claim is made regarding its physical completeness.

The target is parametrized with the following damage model as a function of the fatigue cycle *n*, with the initial condition $D(n = 0) = 0$.

$$D(n+1) = D(n) + \lambda(1 - D(n))^{\beta}. \tag{34}$$

With $D(n) \in [0,1]$, $\lambda > 0$ is the effective damage rate [cycle−1], $\beta \geq 1$ controls how damage slows as failure is approached.

Let *y(n)* be a continuous quality signal describing the quality degradation through the fatigue test. We model *y(n)* linearly with *D(n)*

$$y(n) = \kappa D(n). \tag{35}$$

The tokens entering the decoder $\mathbf{Y}^{(1)} = \boldsymbol{\lambda}$; $\mathbf{Y}^{(2)} = \boldsymbol{\beta}$; $\mathbf{Y}^{(3)} = \boldsymbol{\kappa}$ have two features: value and variable identifier. The physics-informed head predicts the values $\hat{\lambda}, \hat{\beta}, \hat{\kappa}$ from the blanked embedded vectors, using only the embedded variable identifier. To bound the predictions, the constraints are enforced via smooth reparameterization through a final Softplus layer.

$$\lambda = \mathrm{Softplus}(\hat{\lambda}). \tag{36}$$

$$\beta = \mathrm{Softplus}(\hat{\beta}). \tag{37}$$

$$\kappa = \mathrm{Softplus}(\hat{\kappa}). \tag{38}$$

Using equations (32)(33), the simulator produces the estimate $\hat{y}(n)$, with $n = 1, \ldots, N_s$, with N_s being the size of the supervisory signal.

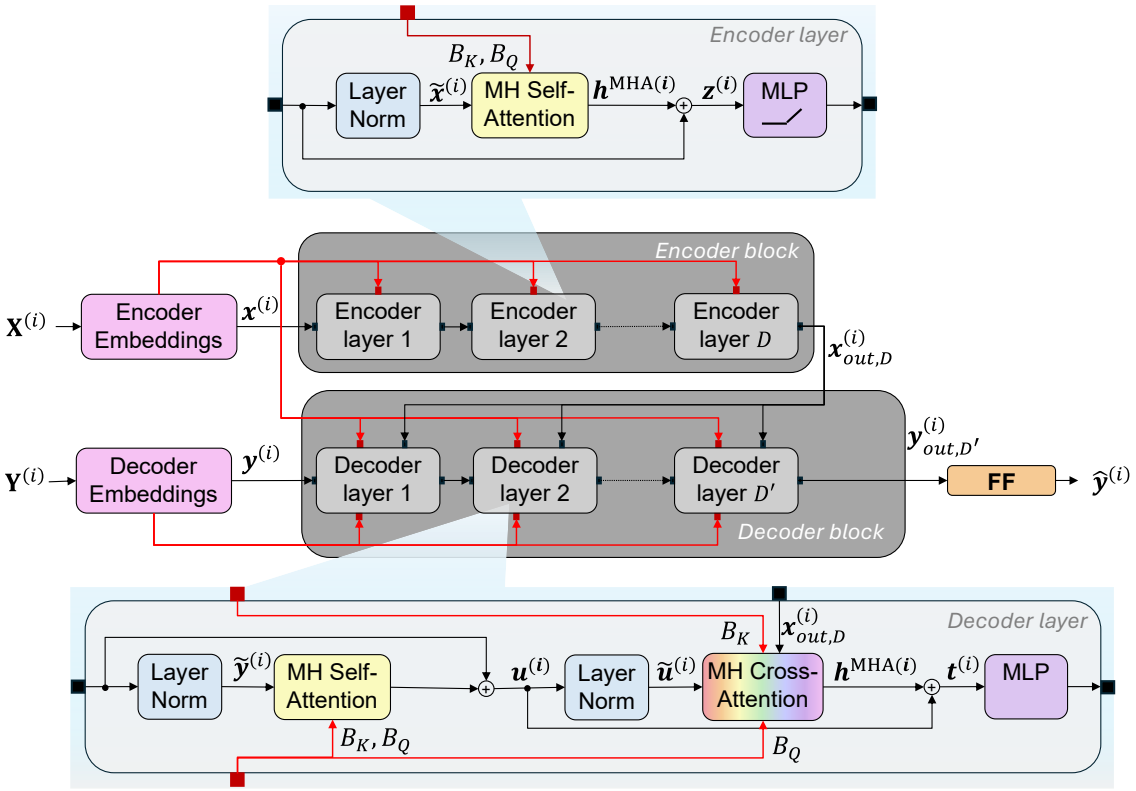


Figure 1: block diagram of the model architecture. It is an encoder-decoder transformer in pre-norm configuration. The transformer predicts all outputs at once, like Informer and Spacetimeformer.

4. Case Study: Multilayer Printed Circuit Boards

Multilayer Printed Circuit Boards (PCBs) are complex electronic substrates composed of multiple conductive layers separated by insulating materials and interconnected through dense networks of copper-filled micro-vias (Fig. 2). Owing to their high aspect ratio and repeated fabrication across layers, micro-vias are among the most reliability-critical features in advanced PCB manufacturing. Small variations introduced at different stages of the process chain can accumulate, leading to premature failures, making PCBs a representative example of manufacturing systems in which end-to-end process interactions determine final product performance.

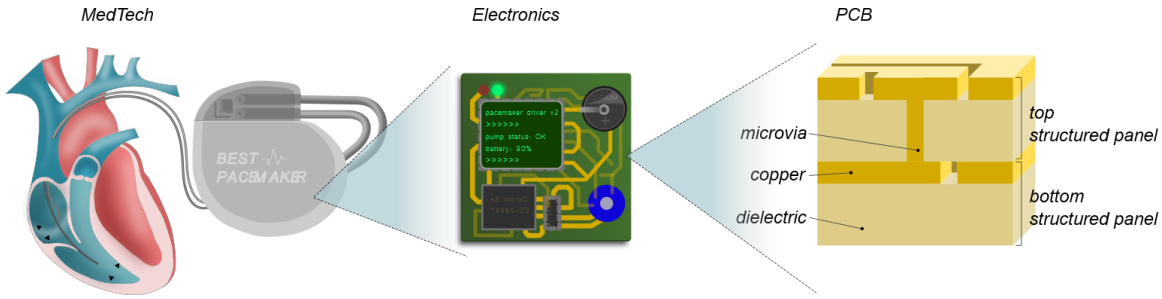


Figure 2: reliable printed circuit boards (PCBs) are the backbone of crucial biomedical devices, such as pacemakers.

4.1. PCB manufacturing process

We focus on the PCB fabrication steps most critical to reliability, namely those associated with the formation of microvias. The input material for subsequent PCB fabrication consists of core panels, i.e., double-sided Cu-cladded inner layers in panel form. A core comprises a rigid dielectric substrate (e.g., polyimide, liquid crystal polymer) with copper on one or both sides. During fabrication, outer layers in the form of single-sided Cu-cladded substrates are sequentially stacked by adhesives (epoxy or acrylic base), with micro-vias introduced after each stacking step. Consequently, micro-via formation constitutes a recurring sequence of operations throughout the PCB panel manufacturing process. A single sequence typically comprises the following steps:

1. Micro-etching: A mild etching step is applied to roughen the copper surface and promote adhesion (Fig. 3a). Panels are sprayed with a solution of potassium peroxysulfate (KHSO5) and sulfuric acid (H2SO4). The temperature is controlled to limit the etch rate. This process ensures the ideal surface roughness for adhesion while also removing contaminants and surface oxides.
2. Lamination: Outer layers are stacked with adhesives in between (Fig. 3b). The stack is placed into a vacuum hydraulic press and undergoes several stages of pressure and temperature. Firstly, at low pressure, the temperature rises (Temperature dependent on the adhesive type), and the adhesive melts into a liquid, filling all the gaps in the structured panel beneath and pushing air bubbles out. Then, pressure is

applied to ensure a uniform thickness and a void-free bond. Finally, the adhesive has fully sealed the panels, and the stack is slowly cooled while still under pressure to prevent warping due to internal stress.

3. Laser drilling: Blind holes are ablated to a controlled depth within the substrate (Fig. 3c). The advantage of laser, compared to mechanical drills, is in the smaller achievable diameter, typically below 150 μm, and execution speed, around 1000 holes/minute. Usually, two types of sources are used. (1) CO2 lasers for punching fast through the dielectric with glass fiber reinforcements, but reflect off shiny copper. (2) UV lasers with the shorter wavelength for both copper and organic dielectric with higher precision, thanks to. A drawback of laser drilling is the vaporization of organic components, known as laser soot, which deposits on the bottom of the hole.
4. Plasma cleaning (desmear): The drilled cavities are cleaned using plasma to remove debris and laser soot (Fig. 3d). The process happens in a vacuum chamber with controlled gas mixture flow of typically O2 and CF4. The gas is ionized into plasma by a high-frequency electromagnetic field (typically radio frequency, i.e., 13.56 MHz). While oxygen radicals (O*) and fluorine radicals (F*) handle organic epoxy residuals.
5. Galvanic (Fig. 3e): the via is filled with copper, electrically connecting the top and bottom electrodes. First, a thin seed layer of copper that uniformly covers the via walls is grown through a slow chemical dip. Then the via is filled with copper deposited by electrolytic plating, allowing greater thickness. The process takes place in a bath of copper sulfate (CuSO4). The panel is connected to the negative terminal of the power supply, flooding the PCB with electrons. The positive terminal is connected to the copper source, which releases Cu2+ ions into the solution. The Cu2+ ions finally drift and deposit on the negatively charged copper of the panel, filling the via.

Despite the repetitive nature of this sequence, processing parameters and conditions vary depending on the specific core or subsequent layers involved, as well as the broader manufacturing context. These variations significantly influence the resulting panel quality,

the prediction of which constitutes the primary objective of the proposed computational model.

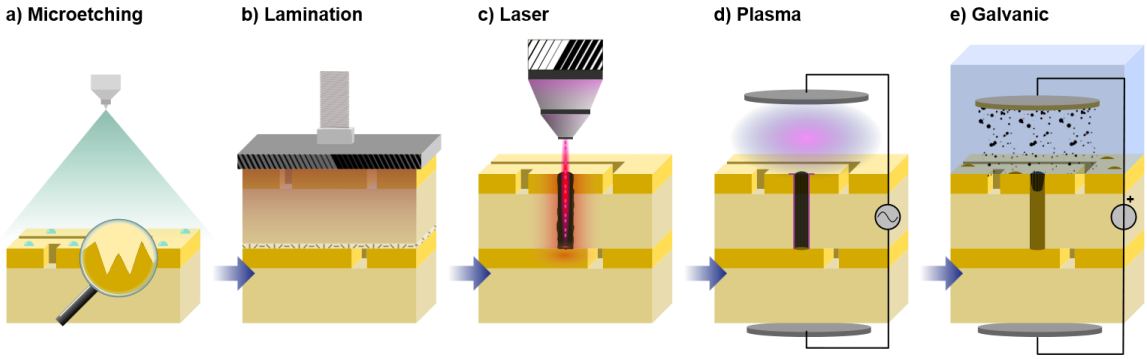


Figure 2: Visualization of a microvia fabrication sequence. The sequence starts by activating the surface with microetching (a) before binding the next panel in the lamination process (b). Next, the via is drilled (c), cleaned (d), and filled with copper (e).

4.2. PCB quality assessment

The interconnect stress test (IST), defined by the industry standard IPC-TM-650 Method 2.6.26 for printed circuit boards (PCBs), is used here to assess the reliability of micro-vias (and multilayer interconnects) by continuously monitoring electrical resistance during cyclic thermal loading. The test is performed on dedicated PCB coupons that undergo repeated Joule-heating cycles until either a failure is detected or the predefined maximum number of cycles is reached.

During each cycle, the applied current induces temperature ramps from approximately 25°C to 210°C (Fig. 3a), generating thermomechanical stress within the copper-filled micro-vias. In low-quality PCBs, this can trigger progressive damage, ultimately leading to micro-fractures (Fig. 3b), which manifest as increases in electrical resistance over time (Fig. 3c). The IST response reflects the cumulative effect of variations introduced throughout the entire manufacturing process chain, rather than the outcome of any single fabrication step. From a modeling perspective, IST curves provide a particularly informative supervisory signal: they are highly sensitive to subtle process variations,

exhibit strong nonlinearity, and encode both gradual degradation and abrupt failure events. If the manufacturing process produces suboptimal structures, these effects are amplified during testing, allowing the IST to act as an integrated measure of process quality.

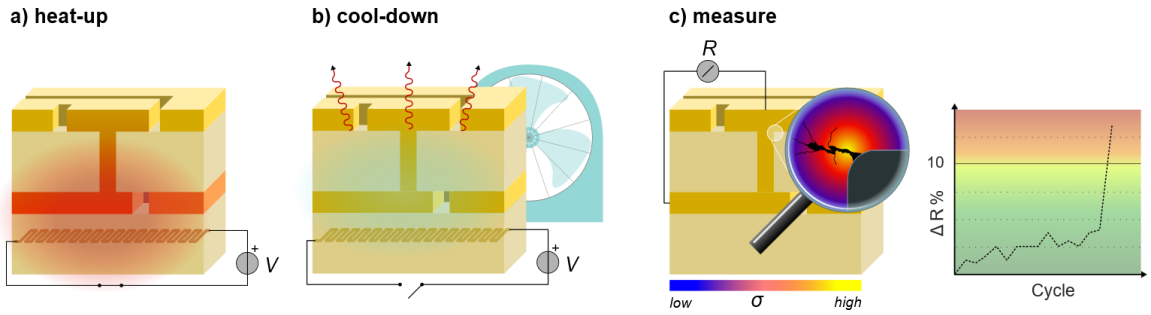


Figure 3: Visualization of the IST procedure. In the (a) step, the structure is heated up to the target temperature by applying the current to the heating line. Then, the structure is cooled down by air fans (b). Finally, the electrical resistance is measured on the sensing line (c). Over time, damage from thermally induced stress σ will be detectable by the increase in resistance. An increase of over 10% is generally classified as a failure of standard industrial protocols.

5. Data

We consider two complementary datasets to evaluate the proposed framework under both controlled and realistic conditions. A synthetic dataset is used to systematically analyze model behavior under controlled nonlinearity, missing data, and limited sample sizes. A real-world industrial dataset is then employed to assess performance in a practical manufacturing setting characterized by complex process chains, heterogeneous sequence lengths, and incomplete observations. Together, these datasets allow us to disentangle representational capacity from data-related effects and to evaluate the robustness of the model under conditions commonly encountered in manufacturing applications.

5.1. Real-world dataset

We evaluate the proposed framework on an industrial dataset provided by the printed circuit board manufacturer Dyconex AG (Switzerland). The dataset captures real manufacturing process chains for multilayer PCBs and includes both process parameters recorded during fabrication and corresponding reliability test measurements. Compared to the synthetic benchmark, this dataset exhibits higher structural complexity, substantial variability in sequence length, and a significant fraction of missing values, reflecting realistic industrial conditions. Overall, the dataset comprises 1920 input sequences. The sequence length varies from 111 to 1023, with an average of 809 over the entire input dataset. Shorter sequences are padded to the maximum length with NaNs in the value feature and zeros in all others, since zero is the embedding index reserved for padding. Excluding padding, input data have 7.2% of missing values on average, with worst cases up to 25%. Missing values are set to NaN while leaving other features intact to distinguish them from padding.

5.1.1 Input data

The manufacturing process chain is described using the following features that uniquely identify the position of each parameter within the process, its value, and identity:

- Feature #1 (process): describes class of operation (e.g., laser, plasma);
- Feature #2 (occurrence): counts how many times a given process class has appeared and implicitly encodes the PCB layer index
- Feature #3 (step): distinguishes sub-operations within a process. For example, consider laser drilling: processing one panel often requires flipping it to drill the other side; hence, two steps are needed.
- Feature #4 (variable): ID of the process parameter, e.g., “plasma RF voltage amplitude”.
- Feature #5 (value): numerical value of the parameter
- Feature #6 (order): the temporal order calculated from the timestamps. This is needed to ensure causality inside the attention block, i.e., tokens can only be influenced by past ones.

- Features #7-11 (timestamp): time components of the logging files, split into years, months, days, hours, minutes. These features describe seasonal trends in the data, as well as periodic maintenance effects and sensors' drift.

Features #1-3 are necessary to disambiguate repeated parameters and represent the cyclic structure of the manufacturing pipeline. They describe the *context* of a given process parameter, i.e., the specific location in the sequence of the manufacturing process chain. Feature #4 is the *value* of the parameter. The value alone would be sufficient to describe the input in a tabular setting, but in our case, both the value and the context are needed.

5.1.2 Output data

The supervisory signal consists of interconnect stress test (IST) measurements, which are time series. Nevertheless, time is not the abscissa of the series; the cycle number is. Each target sequence includes positional indices and timestamps corresponding to the measurement apparatus, while the value feature records the normalized resistance variation. Like for the input, the target is described by several features:

- Feature #1 (cycle number): the sequential dimension of the time-series. As the resistance is measured after each cycle, this is the natural positional feature of the IST.
- Feature #2 (variable): on the test structure, two possible circuits are testable, here called Sense A/B, which differ by the via design (stacked versus staggered).
- Feature #3 (value): numerical value of the resistance increase measured by the IST apparatus
- Features #4-8 (timestamp): creation date of the IST log file.

Missing values are present in both input and target sequences and are retained as NaNs to allow explicit handling through attention masking.

5.2. Synthetic dataset

To systematically study the behavior of the proposed model under controlled conditions, we construct a synthetic dataset that emulates idealized manufacturing process chains and their associated quality supervisory signals in a simpler setting. The dataset is designed to isolate representational and generalization effects while allowing precise control over nonlinearity, noise, and data corruption.

The supervisory signal is generated using the Ishigami function (Ishigami et al., 1990), a standard benchmark in sensitivity analysis known for its strong nonlinearity and interaction effects.

$$\mathrm{y} = \sin(\mathrm{x}_1) + \mathrm{a}\sin^2(\mathrm{x}_2) + \mathrm{b}\mathrm{x}_3^4\sin(\mathrm{x}_1) \tag{37}$$

where x_1, x_2 and x_3 represent latent process variables.

Rather than providing these variables directly as scalar inputs, each target is generated by a sequence of latent process variables that take on different orders and values. Input tokens have the following features.

- Feature #1 numeric value
- Feature #2 (variable) categorical latent variable identifier from the set $\{x_1, x_2, x_3\}$.
- Feature #3 (position) simplified context of the process chain, described only by a positive integer.
- Feature #4 (noise) random independent noise $\sim \mathcal{N}(0, \sigma^2)$ is included for a realistic stochastic description.

The target is likewise represented as a list of tokens with two features

- Feature #1 numeric value
- Feature #2 cycle number.

The synthetic dataset is generated by choosing a maximum input list size L_x and generating input tokens $\mathbf{X}^{(1)}, \mathbf{X}^{(2)}, \ldots, \mathbf{X}^{(L_x)}$ with a predefined variable feature order, e.g. $[\mathrm{x}_1, \mathrm{x}_2, \mathrm{x}_3]$. The value $v^{(i)}$ and noise $\eta^{(i)}$ features are sampled from the distributions $U(0,1)$ and $U(0,\gamma)$, where $0 \leq \gamma \leq 1$ is the noise strength. The position feature is simply the order $1,2,\ldots,L_x$. To produce the output with the Ishigami function, token values of the same variables are aggregated into effective $\tilde{\mathrm{x}}_1, \tilde{\mathrm{x}}_2, \tilde{\mathrm{x}}_3$. Each token contribution to the respective effective variable is

$$c^{(i)} = v^{(i)}\left(i + \epsilon\eta^{(i)}\right) \cdot \mathrm{boost}^{(\mathrm{i})} \tag{38}$$

Where $\mathrm{boost}^{(\mathrm{i})} = 1_{\{i \in \mathcal{B}\}}K$ is a position-specific multiplier (by a factor K) to further inject a sparse positional dependence of the contribution. The contributions are first averaged and rescaled in $[-\pi, \pi]$, obtaining $\tilde{\mathrm{x}}_1, \tilde{\mathrm{x}}_2, \tilde{\mathrm{x}}_3$. The effective variables are used to produce a latent $\tilde{y}^{(i)}$ from (37). To produce the target tokens $\mathbf{Y}^{(1)}, \mathbf{Y}^{(2)}, \ldots, \mathbf{Y}^{(L_y)}$, a maximum sequence size L_y is chosen. The cycle features are simply the indices $1, \ldots, L_y$. The values $u^{(i)}$ are obtained from the following polynomial relation

$$u^{(i)} = \tilde{y}^{(i)}\left(c_1\left(\rho^{(i)}\right)^2 + c_2\left(\rho^{(i)}\right)^4 + c_3\left(\rho^{(i)}\right)^5\right) + \zeta^{(i)} \tag{39}$$

where $\rho^{(i)} = i/L_y$, a,b and c are real constants and $\zeta^{(i)} \sim N(0, \sigma_n^2)$.

The parameters used to generate the synthetic dataset are provided in the Appendix.

While the synthetic dataset is fully observed by construction, the real-world dataset provides a challenging testbed for assessing model robustness due to incomplete data and limited sample availability.

Table 1 summarizes the main characteristics of the synthetic and real-world datasets. The comparison highlights the complementary roles of the two datasets: the synthetic benchmark provides full control over sequence length, dimensionality, and data corruption, while the real-world dataset reflects the scale, sparsity, and heterogeneity of industrial manufacturing data.

	Synthetic	Real-World
Input features	value, variable, position, noise	value, process, occurrence, step, variable, time
Target features	value, position	value, variable, position, time
Max input sequence length	20	1290
Max target sequence length	20	400
Number of input variables	3	290
Number of target variables	1	2
Number of samples	16000	1900
Max missing values per sequence	0%	25% (excluding padding)

Table 1: Summary of the synthetic and real-world datasets used in this study. The synthetic dataset enables controlled analysis of representation learning under nonlinearity, missing values, and limited sample sizes, while the real-world dataset reflects realistic industrial conditions characterized by long process chains, heterogeneous sequence lengths, and substantial sparsity.

5.3. Rarity-based slicing

Both datasets exhibit strongly imbalanced target distributions, with rare events corresponding to critical but infrequent outcomes. In the real-world dataset, early or abrupt failures occur rarely due to the overall stability of the manufacturing process. The synthetic dataset is constructed to reproduce a similar imbalance, ensuring consistency across experimental settings. To account for this skew and to enable a meaningful evaluation of

model generalization, we assign each sample a rarity score $\in [0,1]$ based on the terminal value of its target sequence. Samples are then grouped into bins according to this score, and an 80–20 train–test split is performed independently within each bin. This stratified procedure ensures that both common and rare cases are represented in the training and test sets, preventing performance estimates from being dominated by the majority regime.

6. Implementation

6.1. Framework

We implemented both the model and data processing in Python. The transformer was implemented by adapting the Spacetimeformer, using PyTorch and PyTorch Lightning for training. The experiments have been carried out on the Euler cluster using the RTX4090 GPU. We used the Optuna framework for hyperparameter search through Bayesian optimization.

6.2. Feature embeddings

The features of the two datasets are fused into the d-dimensional representation processed by the transformer by applying specific feature-wise maps $\boldsymbol{\varphi}$ to the input tokens. We have used four different types of maps:

1. Learnable embeddings: lookup tables composed of d-dimensional row vectors whose entries are learnable values. For categorical data, a dictionary is assembled to map each category to an integer index. Each index corresponds to its d-dimensional row vector.
2. Sinusoidal embeddings. They are fixed d-dimensional representations of integers, often representing positional features, as in the original Transformer. For the integer k, the corresponding d-dimensional encoded representation PE is a vector such that even entries are produced by the following map

$$PE_{2i} = \sin\left(\frac{k}{10000^{\frac{2i}{d}}}\right) \tag{40}$$

While odd entries by

$$PE_{2i+1} = \cos\left(\frac{k}{10000^{\frac{2i}{d}}}\right) \tag{41}$$

Note that the entries $i = 0, \dots, d-1$.

3. Time2Vec: an embedding map tailored on timestamps. Consider a timestap composed by J components (e.g., "DD", "HH", "MM" for J=3). The t2v maps construct a K-dimensional representation of the τ component as follows

$$t2v[\tau] = \begin{cases} \omega_i\tau + \phi_i \text{ if } i = 0 \\ \sin(\omega_i\tau + \phi_i) \text{ if } 0 < i \leq K-1 \end{cases} \tag{42}$$

Where ω_i, ϕ_i for $i = 0 \dots K-1$. Then, the J components are summed together and projected to d by a linear map $W_t \in R^{d\times K}$. The final embedded vector is given by

$$W_t\left(\sum_{i=1}^{J} t2v[\tau]\right) \tag{43}$$

4. Linear maps: used to map continuous values to the desired dimension d

The precise coupling of embedding maps and dataset features is available in the Appendix.

6.3. Training

The model parameters are optimized by minimizing a mean squared error (MSE) loss between predicted and target sequences. The training is implemented in the standard minibatch gradient descent algorithm, with parameters updated at each step, i.e., after each minibatch is processed. The procedure repeats for a predefined maximum number of epochs. To discourage degenerate attention patterns, in which a small subset of tokens

dominates the attention distribution, we include an entropy-based regularization term in the attention scores.

Let $\mathcal{E}_j^{(b)}$ be the batch-averaged attention score entropy of the jth attention block considering both self- and cross-attention layers, for the b-th batch. Let $\alpha^{(j,l)}$be the $L_q \times L_k$ attention score matrix for the j-th attention block and l-th sequence in a batch of B samples. L_q, L_k are the query and key sequence lengths. The batch-average score entropy is given by

$$\mathcal{E}_j^{(b)} = \frac{1}{B}\sum_{l=1}^{B}\frac{1}{L_q}\sum_{i=1}^{L_q}\sum_{k=1}^{L_k}\alpha_{ik}^{(j,l)}\log\alpha_{ik}^{(j,l)} \tag{44}$$

Let $\widehat{\boldsymbol{y}}^{(i,j)}$ be the model output for the token pairs $(\mathbf{X}^{(i,j)}, \mathbf{Y}^{(i,j)})$ and $\boldsymbol{y}^{(i,j)}$ Its target value is provided by the supervisory signal, where i is the list index and j is the covariate-pair index within the minibatch. The batch mean squared error for the b-th batch is given by

$$\mathrm{MSE}^{(\mathrm{b})} = \frac{1}{B}\sum_{j=1}^{B}\frac{1}{L_y}\sum_{i=1}^{L_y}\left(\widehat{\boldsymbol{y}}^{(i,j)} - \boldsymbol{y}^{(i,j)}\right)^2 \tag{45}$$

By including the entropy as a penalty term, the training objective becomes

$$\mathcal{L} = \mathrm{MSE}^{(\mathrm{b})} + \lambda\sum_{j\in\mathcal{A}}\frac{1}{\mathcal{E}_j^{(b)}} \tag{46}$$

Where $\mathcal{A}$ denotes the set of attention blocks of the model and λ controls the strength of the regularization. This term penalizes overly concentrated attention distributions and promotes more balanced information flow across tokens. Training is performed using five-fold cross-validation with the AdamW optimizer (Loshchilov et al., 2019). For each fold k, model parameters w are obtained by solving

$$\widehat{\mathbf{w}}_{\mathrm{k}} = \underset{\mathbf{w}}{\mathrm{argmin}}\mathcal{L}. \tag{47}$$

The resulting models can be used collectively to estimate validation and test statistics, or a single model can be selected based on a predefined criterion, such as best validation or test performance.

6.4.Hyperparameters

The transformer model is defined by the following configuration options: model dimension d; attention dot product dimension d_{QK}; encoder/decoder number of heads H, H'; number of encoder/decoder layers D, D'. In this study, we set $H = H'$ for simplicity, the MLPs are specified by an additional hyperparameter d_h used by two linear maps $\boldsymbol{W}_1 \in R^{d \times d_h}$ and $\boldsymbol{W}_2 \in R^{d_h \times d}$. The MLP function is defined as $f^{\mathrm{MLP}}: X \in R^d \rightarrow \sigma(\sigma(X\boldsymbol{W}_1)\boldsymbol{W}_2) \in R^d$, where σ is the GeLU activation. Dropout is further applied to every layer of the model and is controlled by a single hyperparameter "dropout". Further learning hyperparameters are the learning rate and the entropy regularization constant λ. Additional settings of AdamW optimizers are the PyTorch default: betas = (0.9, 0.999), weight decay = 0.01. All hyperparameters are optimized by selecting the Optuna trial that minimizes the validation loss. In the Appendix, we report a table with the search space and optimal values.

7. Results

In this section, the most important findings of this study are presented. We first show the case-study predictions in sequence-to-sequence and physics-informed modes. Then, the case-study is further used to demonstrate the application of curriculum learning to improve predictions as measurements are available. Following, we study the effect of data corruption on the synthetic dataset. Finally, the benchmarks are reported.

7.1.Case-Study

7.1.1 Sequence to Sequence and Physics-Informed Head

In some manufacturing settings, partial physical knowledge of the target process is available and can be leveraged to improve interpretability or reduce output dimensionality. Rather than predicting raw measurement sequences directly, it can be advantageous to infer

a small set of physically meaningful parameters from which the observable signal can be reconstructed.

To demonstrate this capability, we couple the manufacturing representation learned by the transformer with a simple physics-based damage model that describes the evolution of damage under cyclic thermal loading and uses a lightweight simulator to generate the corresponding resistance curve. Instead of predicting the full IST sequence, the transformer is trained to infer the physical variables fed to the simulator, which then rolls out the target signal.

This physics-informed head uses the same encoder and decoder architecture as in the sequence-to-sequence setting, differing only in the form of the supervisory signal. While this approach yields slightly less accurate predictions than direct sequence prediction, it requires only a small number of output tokens and produces interpretable quantities directly related to the underlying degradation mechanisms. The results are shown in Figure 4.

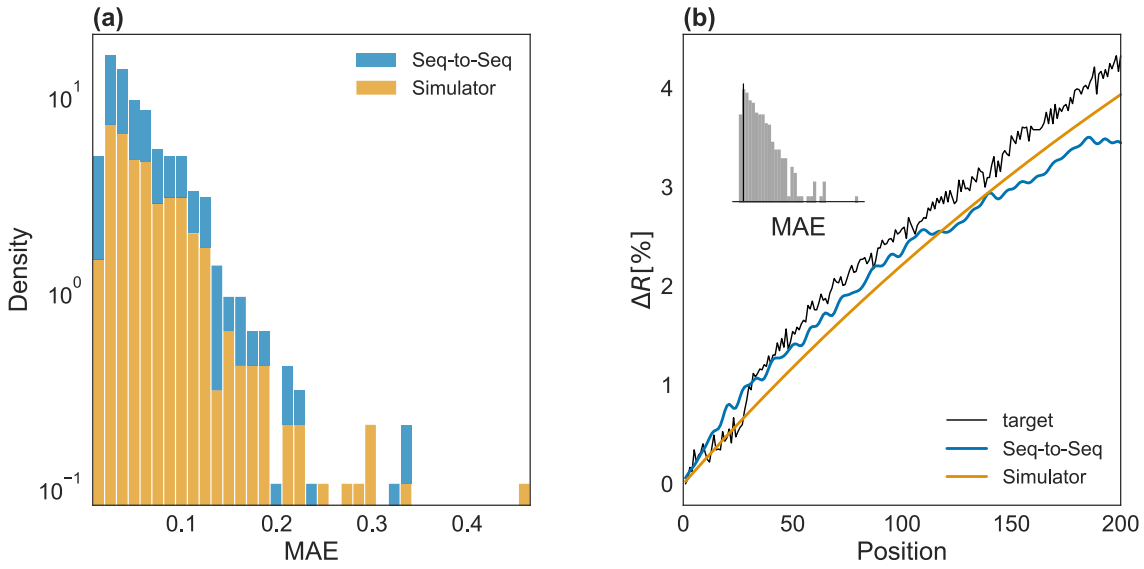


Figure 4: Comparison between direct sequence-to-sequence prediction and a physics-informed output head on the real-world PCB dataset. (a) Distribution of test-set mean absolute error (MAE) for both inference modalities. The physics-informed head predicts a small set of physical parameters, which are then rolled out by a lightweight simulator,

resulting in slightly reduced predictive accuracy compared to direct sequence prediction. (b) Representative test-sample prediction from the dominant error bin. While the physics-informed approach produces smoother, less accurate curves, it infers physically interpretable quantities from only three output tokens rather than the full target sequence. These results illustrate the trade-off between predictive accuracy and interpretability that arises from changing the supervisory signal while keeping the underlying manufacturing representation fixed.

7.1.2 Curriculum learning for rare samples

Rare but critical events pose a challenge in many manufacturing applications, as they are underrepresented in the available data yet carry disproportionate practical importance. In the context of PCB reliability testing, such cases correspond to premature failures characterized by shorter sequences and abrupt increases in resistance.

Rather than introducing specialized architectures or loss functions, we use this case study to illustrate how the proposed transformer framework can be adapted to such scenarios with minimal modifications. Specifically, we consider two simple strategies that exploit the flexibility of the sequence-based representation.

First, an auxiliary classification head is added to predict whether a target sequence corresponds to a valid or truncated measurement. This provides an additional supervisory signal in regions where the primary regression target is padded, allowing the model to better distinguish early failure patterns. Second, we adopt a curriculum learning strategy in which partial target sequences are progressively revealed during training. At each epoch, only measurements up to a randomly sampled position are provided, while the remainder of the sequence is left blank.

Beyond improving learning of rare samples, this training procedure naturally enables online inference: as new measurements become available, predictions can be updated in real time without retraining the model. Figure 5 illustrates how incorporating early

measurements substantially improves prediction accuracy for a rare failure case, highlighting the practical utility of this approach.

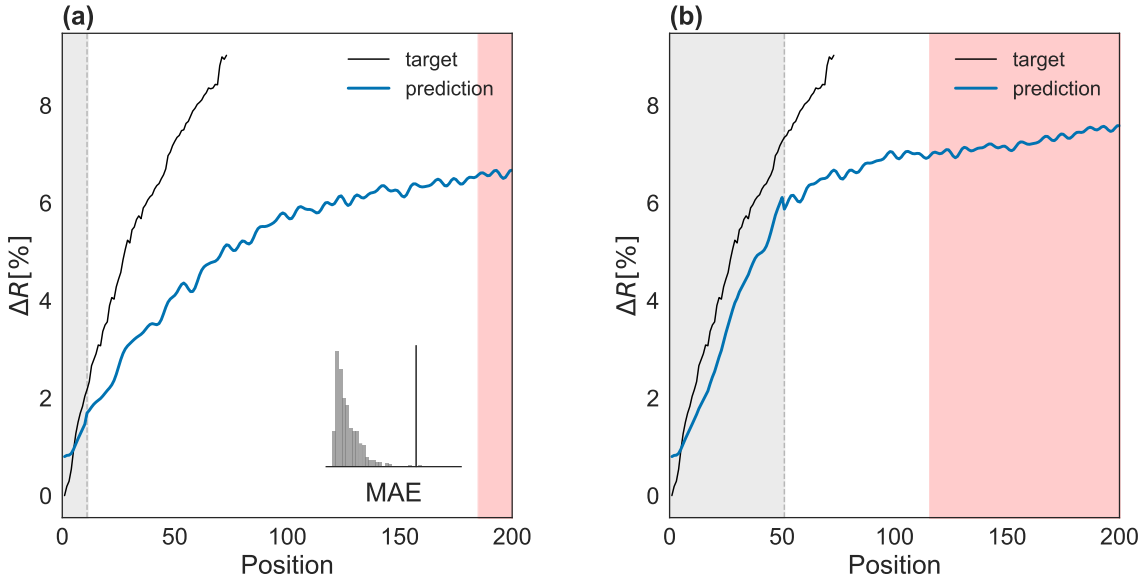


Figure 5: Adaptive prediction for a rare failure case using curriculum learning and online inference. The shown sample corresponds to a high-rarity test instance characterized by premature failure and a steep increase in resistance. (a) Prediction obtained when only the first 10 measurements are provided. (b) Updated prediction after incorporating the first 50 measurements. As additional measurements become available (gray region), the model progressively refines its prediction and localizes the imminent breakdown region (red region) more accurately. This behavior demonstrates how the sequence-based representation naturally supports online inference without retraining.

7.2.Missing values

To evaluate robustness under missing data, we study performance degradation as a function of controlled input corruption. In the synthetic dataset, missing values are introduced by replacing token values with NaNs according to a Bernoulli process with probability p_{blank}. This allows us to isolate the effect of missing information on both representation learning and inference.

Since missing values can affect training and inference in fundamentally different ways, we consider two complementary experimental settings. In the first setting, models are trained on fully observed data and evaluated on corrupted test inputs, probing how well a learned representation can be exploited when information is partially unavailable. In the second setting, models are trained on corrupted data and evaluated on fully observed test inputs, assessing whether explicit masking prevents missing values from biasing the learned representation.

1. Missing values at inference: Models are trained on the original, uncorrupted dataset and evaluated on test inputs with increasing levels of missing values. In this setting, NAIM prevents queries to missing tokens, and performance degradation reflects how much the learned representation relies on complete information.
2. Missing values during training. Models are trained on datasets corrupted with different values of p_{blank} and evaluated on an uncorrupted test split. This setting isolates the effect of missing data on representation learning and tests whether NAIM can prevent spurious correlations introduced by imputation or missing-value bias.

The results are reported in Fig. 3.

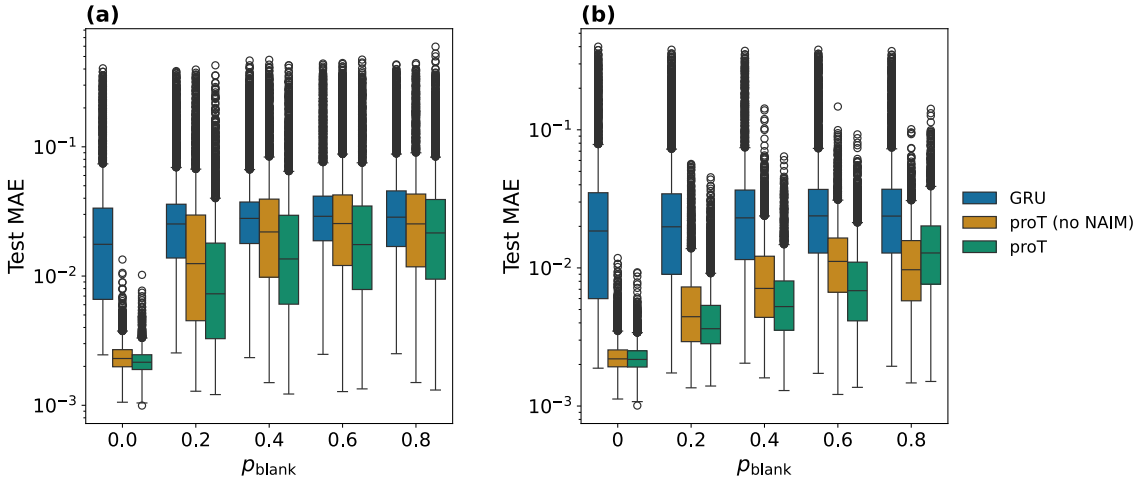


Figure 2: Performance degradation under controlled missing-value corruption on the synthetic dataset. Test mean absolute error (MAE) is reported as a function of the blanking

probability p_{blank}. (a) Models trained on fully observed data are evaluated on corrupted test inputs, measuring sensitivity to missing information at inference time. (b) Models trained on corrupted data are evaluated on uncorrupted test inputs, isolating the effect of missing values during representation learning. Results are shown for the proposed transformer with NAIM, the same architecture without NAIM, and a GRU baseline. Error bars denote variability across cross-validation folds.

7.3. Results

- When models are trained on fully observed data and evaluated on corrupted inputs (a), prediction error increases rapidly even for small p_{blank} indicating that the learned representation relies on the availability of complete information. In this regime, NAIM provides limited benefit.
- When models are trained on corrupted data and evaluated on fully observed inputs (b), NAIM substantially mitigates performance degradation across a wide range of blanking probabilities, demonstrating its effectiveness in preventing missing values from biasing representation learning.
- For extreme corruption levels ($p_{\text{blank}} \approx 0.8$), the advantage of NAIM diminishes, and transformer performance converges toward that of recurrent baselines

7.4. Data Efficiency

Transformers are often regarded as data-intensive models, raising concerns about their applicability in manufacturing settings where labeled data are limited. To examine this issue, we study how training duration and dataset size jointly affect generalization performance when the training data remain representative of the test distribution.

Using the synthetic dataset, we train the same model for 200, 400, and 800 epochs on progressively smaller subsets of the original training split, containing 12’000, 1’200, and 120 samples, respectively. All models are evaluated on the same held-out test split, and performance is measured using mean absolute error (MAE).

Because the different training subsets are obtained by subsampling, they do not necessarily share the same support as the test distribution. In particular, rare target values may be underrepresented or entirely absent in small training sets. To interpret the observed performance trends, it is therefore essential to analyze the distribution of target values in both training and test splits.

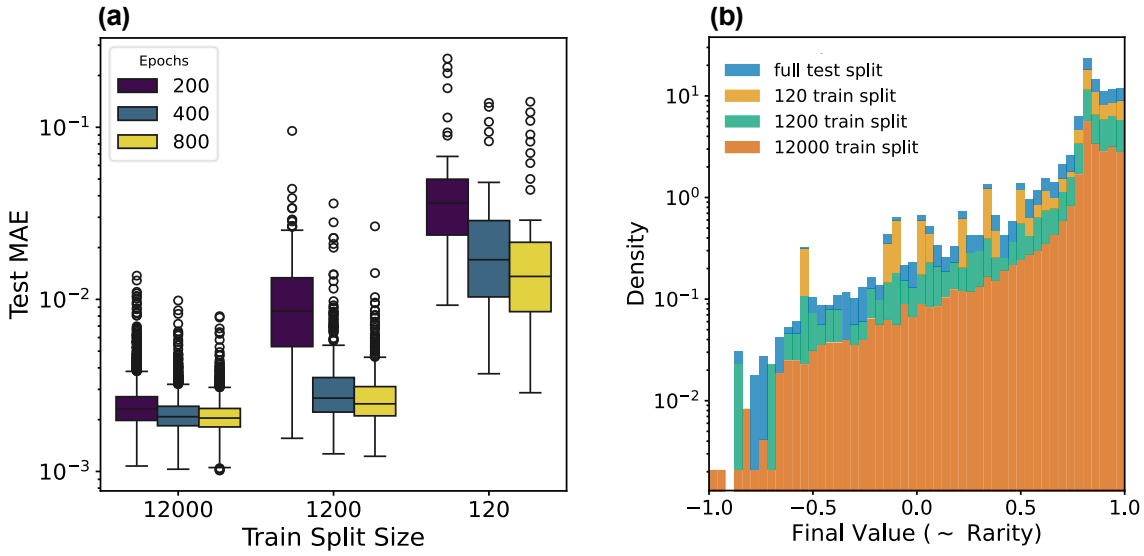


Figure 3: Effect of dataset size and training duration on generalization performance using the synthetic dataset. (a) Test mean absolute error (MAE) for models trained on subsets of the training data with different sizes and for increasing numbers of training epochs. (b) Kernel density estimates of the terminal target value for the corresponding training subsets and the test split, illustrating the degree of distributional overlap. The results highlight the role of representative coverage, rather than dataset size alone, in determining generalization performance.

7.5. Results

- When the training subset provides sufficient coverage of the test distribution (12’000 and 1’200 samples), increasing the number of training epochs leads to consistent improvements in test performance, indicating that the transformer can extract useful representations even from relatively small datasets.

- For the smallest training subset (120 samples), performance does not improve with longer training, reflecting poor overlap between training and test distributions rather than insufficient optimization.
- These results suggest that data representativeness, rather than dataset size alone, is the primary factor governing generalization in low-data regimes.

7.6.Benchmarks

We benchmark the proposed transformer-based model against commonly used sequence-to-sequence architectures in manufacturing and time-series modeling, namely long short-term memory networks (LSTM), gated recurrent units (GRU), and temporal convolutional networks (TCN). As a non-sequential baseline, we also include a multilayer perceptron (MLP) with mean pooling over the input sequence.

To ensure a fair comparison across architectures, all models operate on the same embedded input representation and differ only in their sequence modeling components. Model-specific hyperparameters are optimized using Optuna (Akiba et al., 2019) with a fixed trial budget of 20 trials per baseline model and 50 trials for the transformer, reflecting its larger parameter space. The optimization objective is the mean absolute error (MAE) on the validation set, averaged across five cross-validation folds.

After hyperparameter selection, the best-performing configurations are evaluated on the held-out test splits of both datasets. Table 2 reports test-set MAE and R2 scores together with the corresponding model sizes. In addition to overall performance, we include a variant of the transformer without NAIM to isolate the effect of explicit missing-value masking.

	Synthetic DS			Real-World DS		
Model	Size	Test MAE ↓	Test R2 ↑	Size	Test MAE ↓	Test R2 ↑
Transformer	1054k	**0.003**±0.001	**0.998**±0.001	384k	**0.066**±0.003	**0.636**±0.019
Transformer (no NAIM)	1516k	**0.003**±0.001	**0.998**±0.001	1856k	0.068±0.003	0.62±0.021
GRU	35k	0.037±0.002	0.426±0.006	654k	0.081±0.004	0.505±0.044
LSTM	370k	0.037±0.001	0.426±0.003	1542k	0.084±0.005	0.4 ± 0.234
TCN	374k	0.08±0.001	0.497±0.019	374k	0.08±0.001	0.497±0.019
Baseline (MLP)	580k	0.041±0.001	0.252±0.008	617k	0.085±0.015	0.484±0.14

Table 2: Benchmark comparison on the synthetic and real-world datasets. All models use the same input embeddings and are tuned with Optuna under a fixed trial budget. Performance is reported on the test splits using mean absolute error (MAE) and coefficient of determination (R2). Model size indicates the number of trainable parameters. The best result for each dataset is in bold.

7.7. Results

- The transformer achieves the best predictive performance on both datasets, outperforming recurrent, convolutional, and pooling-based baselines.
- Explicit masking of missing values through NAIM reduces the number of parameters selected during hyperparameter optimization while preserving top performance, with the largest effect observed on the real-world dataset.
- Recurrent models exhibit limited performance on long and heterogeneous sequences, particularly in the real-world dataset, highlighting the benefit of attention-based representations for process chain modeling.

8. Discussion

The experiments presented above evaluate the proposed framework along complementary dimensions, including robustness to missing data, data efficiency under limited supervision, and adaptability to alternative supervisory signals. This section synthesizes the most relevant insights from these results and discusses their implications for modeling manufacturing process chains under realistic industrial constraints.

8.1. Missing Data and Representation Learning

The missing-value analysis highlights an important distinction between robustness at inference time and robustness during representation learning. While performance degradation is unavoidable when information is removed at inference, explicit attention masking primarily protects the learning process by preventing missing values from biasing internal representations. This effect is particularly relevant in manufacturing settings, where missing data are often systematic rather than random and arise from sensor outages, logging inconsistencies, or asynchronous acquisition.

By preventing missing tokens from participating in attention computations, NAIM ensures that learned dependencies reflect available measurements rather than imputation artifacts. However, the diminishing benefit observed under extreme corruption levels indicates that explicit masking cannot compensate for fundamentally insufficient information content. These results suggest that handling missing values within the attention mechanism is most effective as a safeguard for representation learning, rather than as a means of recovering missing information at inference.

8.2. Data Efficiency and Distributional Coverage

The data-efficiency experiments suggest that transformer-based representations can generalize effectively from limited datasets, provided that the training data sufficiently cover the target distribution. Improvements observed with longer training on smaller

datasets reflect improved representation learning rather than overfitting, as generalization performance strongly depends on distributional overlap between training and test data.

This observation is particularly relevant for manufacturing applications, where collecting large labeled datasets is often impractical, but representative coverage of critical operating regimes can still be achieved. At the same time, the failure of very small datasets to benefit from extended training highlights a fundamental limitation: optimization alone cannot compensate for missing coverage of rare but important outcomes. In this sense, dataset representativeness emerges as a more critical factor than dataset size when deploying attention-based models in low-data regimes.

8.3. Flexibility of the Learned Representation

The case-study experiments demonstrate that the learned manufacturing representation is not tied to a specific output format or supervisory signal. Both the physics-informed head and the curriculum-learning strategy reuse the same encoder–decoder architecture and differ only in how supervision is applied. This separation between representation learning and task-specific supervision enables practical adaptations without redesigning the model architecture.

In particular, replacing direct sequence prediction with a physics-informed output head illustrates how prior physical knowledge can be incorporated to improve interpretability and reduce output dimensionality, at the cost of some predictive accuracy. Similarly, the curriculum-learning strategy shows how the same representation can be adapted to rare-event regimes and naturally supports online inference as new measurements become available. These examples are illustrative rather than exhaustive, and are not intended to establish optimal strategies for physics integration or rare-event learning, but to highlight the flexibility of the sequence-based formulation.

8.4. Scope and Limitations

While the results support the use of transformer-based representations for modeling manufacturing process chains, several limitations should be noted. The present study focuses on supervised learning settings and does not address causal identification or counterfactual reasoning. Moreover, although transformers provide a convenient implementation, the core contribution lies in the tokenization of process parameters and the explicit handling of structural and data-related challenges, rather than in the specific attention mechanism employed.

Future work may explore alternative attention formulations, domain-specific inductive biases, or tighter integration with physical constraints. However, such extensions fall outside the scope of the present study, which aims to establish a general and flexible framework for process chain modeling under realistic industrial conditions.

9. Conclusion

This work introduced a sequence-based framework for modeling manufacturing process chains, representing process parameters as context-aware tokens and learning their interactions via attention mechanisms. By treating entire process chains as sequences rather than isolated stages, the proposed approach captures cross-stage dependencies that are critical for predicting downstream quality and reliability.

Through controlled synthetic experiments and a real-world case study on multilayer printed circuit board manufacturing, we demonstrated that the framework remains effective under realistic industrial constraints, including missing data, limited dataset sizes, and rare failure events. The results further showed that explicit handling of missing values within the attention mechanism protects representation learning, and that data representativeness plays a more important role than dataset size alone in low-data regimes.

Beyond predictive performance, the case-study experiments highlighted the flexibility of the learned representation. The same encoder–decoder architecture can be coupled with alternative supervisory signals, such as physics-informed models or curriculum-based

training strategies, without architectural changes. This separation between representation learning and task-specific supervision enables practical adaptations to different manufacturing objectives.

Overall, the proposed formulation provides a general and flexible foundation for data-driven modeling of manufacturing process chains. While transformers provide a convenient implementation, the central contribution lies in tokenizing process parameters and explicitly addressing structural and data-related challenges inherent to industrial settings. We hope this work encourages further exploration of sequence-based representations for complex manufacturing systems.

Acknowledgments

This work was supported by the Swiss Innovation Agency Innosuisse [project number 105.293 IP-ENG].

Data and Code Availability

The datasets and the implementation used in this study are publicly available at https://github.com/scipi1/proT and https://zenodo.org.

References

Ba, J.L., Kiros, J.R., Hinton, G.E., 2016. Layer Normalization. https://doi.org/10.48550/arXiv.1607.06450

Bahdanau, D., Cho, K., Bengio, Y., 2016. Neural Machine Translation by Jointly Learning to Align and Translate.

Brown, T.B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D.M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., Amodei, D., 2020. Language Models are Few-Shot Learners. https://doi.org/10.48550/arXiv.2005.14165

Caruso, C.M., Soda, P., Guarrasi, V., 2024. Not Another Imputation Method: A Transformer-based Model for Missing Values in Tabular Datasets. https://doi.org/10.48550/arXiv.2407.11540

Du, S., Xu, R., Huang, D., Yao, X., 2015. Markov modeling and analysis of multi-stage manufacturing systems with remote quality information feedback. Comput. Ind. Eng. 88, 13–25. https://doi.org/10.1016/j.cie.2015.06.012

Du, S., Xu, R., Li, L., 2018. Modeling and Analysis of Multiproduct Multistage Manufacturing System for Quality Improvement. IEEE Trans. Syst. Man Cybern. Syst. 48, 801–820. https://doi.org/10.1109/TSMC.2016.2614766

Grigsby, J., Wang, Z., Nguyen, N., Qi, Y., 2023. Long-Range Transformers for Dynamic Spatiotemporal Forecasting.

He, K., Zhang, X., Ren, S., Sun, J., 2016. Deep Residual Learning for Image Recognition, in: 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR). Presented at the 2016 IEEE Conference on Computer Vision and Pattern

Recognition (CVPR), IEEE, Las Vegas, NV, USA, pp. 770–778. https://doi.org/10.1109/CVPR.2016.90

Huang, Q., Shi, J., 2004. Stream of Variation Modeling and Analysis of Serial-Parallel Multistage Manufacturing Systems. J. Manuf. Sci. Eng. 126, 611–618. https://doi.org/10.1115/1.1765149

Kazemi, S.M., Goel, R., Eghbali, S., Ramanan, J., Sahota, J., Thakur, S., Wu, S., Smyth, C., Poupart, P., Brubaker, M., 2019. Time2Vec: Learning a Vector Representation of Time.

Lu, Y., Lu, J., n.d. A Universal Approximation Theorem of Deep Neural Networks for Expressing Probability Distributions.

Mok, S.L., Kwong, C.K., 2002. Application of artificial neural network and fuzzy logic in a case-based system for initial process parameter setting of injection molding. J. Intell. Manuf. 13, 165–176. https://doi.org/10.1023/A:1015730705078

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I., n.d. Language Models are Unsupervised Multitask Learners.

Shewhart, W.A. (Walter A., Deming, W.E. (William E., 1939. Statistical method from the viewpoint of quality control. Washington, The Graduate School, The Department of Agriculture.

Tsai, Y.-H.H., Bai, S., Yamada, M., Morency, L.-P., Salakhutdinov, R., 2019. Transformer Dissection: A Unified Understanding of Transformer's Attention via the Lens of Kernel.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I., 2023. Attention Is All You Need.

Xiong, R., Yang, Y., He, D., Zheng, K., Zheng, S., Xing, C., Zhang, H., Lan, Y., Wang, L., Liu, T.-Y., 2020. On Layer Normalization in the Transformer Architecture.

Zhou, H., Zhang, Shanghang, Peng, J., Zhang, Shuai, Li, J., Xiong, H., Zhang, W., 2021. Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting.

Zhou, S., Ding, Y., Chen, Y., Shi, J., 2003. Diagnosability Study of Multistage Manufacturing Processes Based on Linear Mixed-Effects Models. Technometrics 45, 312–325. https://doi.org/10.1198/004017003000000131

10 Appendix

10.1. Synthetic dataset parameters

Parameter	Symbol	Value
Input list size	L_x	20
Target list size	L_y	20
Noise strength	γ	1.0
Noise weight	ϵ	1E-3
Boost positions	$\mathcal{B}$	{2, 4, 8, 10, 15}
Boost factor	K	5.0
Ishigami coefficients	$a; b$	7; 0.1
Polynomial coefficients	$c_1; c_2; c_3$	0.3 ; 0.4 ; 0.3
Target noise variance	σ_n^2	0

Table 3: Parameters used to generate the synthetic dataset.

Feature-embeddings maps

Maps	Synthetic		Real-World	
	X	Y	X	Y
Learnable embeddings	#2 (variable) #3 (position)		#1 (process) #2 (occurrence) #3 (step) #4 (variable) #6 (order)	#2 (variable)
Sinusoidal embeddings	NA	#2 (cycle number)	NA	#1 (cycle number)
Time2Vec	NA	NA	#7-11 (timestamp)	#4-8 (timestamp)
Linear	#1 (value) #4 (noise)	#1 (value)	#5 (value)	#3 (value)

Table 4: Summary of the embedding maps used for the features of the two datasets.

10.2.Hyperparameter optimization

Hyperparameter	Symbol	Search space	Optimal value

				Synthetic DS	Real-world DS
Transformer	Model dim.	d	50 - 200	190	110
	Att. dot prod. dim.	d_{QK}	100 - 200	100	200
	Num. of heads	H	1 - 3	2	1
	Num. enc. layers	D	1 - 3	2	1
	Num. dec. layers	D'	1 - 3	3	1
	MLP projection	d_h	200 - 600	200	200
	Global dropout	"dropout"	0.0 0.3	0.0	0.0
	Learning rate	η	1E-4 - 1E-4	9E-4	9E-4
	Reg. constant	λ	1E-4 – 1E-2	1E-4	4E-4
Transformer w/o NAIM	Model dim.	d	50 - 200	110	170
	Att. dot prod. dim.	d_{QK}	100 - 200	150	150
	Num. of heads	H	1 - 3	2	2
	Num. enc. layers	D	1 - 3	2	2

Num. dec. layers	D'	1 - 3	3	3
MLP projection	d_h	200 - 600	500	500
Global dropout	"dropout"	0.3	0.0	0.0
Learning rate	η	1E-4 - 1E-4	7E-4	5E-4
Reg. constant	λ	1E-4 – 1E-2	1.2E-4	1.3E-3

Table 5: Hyperparameter optimization results of the transformer architectures.